

AsymAgentMark-TK: Weakly Asymmetric Behavioral Watermarking for LLM Agents

Anonymous Authors
Paper under double-blind review

Abstract—Autonomous LLM agents increasingly make consequential multi-step decisions through planning behaviors such as tool selection, subgoal choice, and embodied action selection. Existing content watermarks attribute generated text, but they do not directly attest to the planning behavior that drives an agent’s execution. AgentMark introduced distribution-preserving behavioral watermarking for this setting, but its concrete symmetric construction requires the verifier to reconstruct the exact behavior distribution used by the embedder. This assumption is brittle under realistic verification: API quantization, temperature changes, model refreshes, and cross-model re-querying may preserve the order of likely behaviors while perturbing probability values enough to break symmetric decoding.

We propose AsymAgentMark-TK, a weakly asymmetric behavioral watermark for LLM agents. The embedder uses full per-step behavior probabilities to preserve the agent’s marginal policy, while the verifier needs only top- k rank order. The construction replaces probability-dependent differential bins with an interleaved binary partition tree over rank-sorted behaviors. Each selected behavior’s rank determines a decoding path, allowing bit recovery from rank positions alone. A fixed-budget HMAC-SHA512 DRBG schedule keeps encoder and decoder synchronized, and deterministic random linear network coding turns variable per-step bits into erasure-tolerant payload recovery.

We evaluate AsymAgentMark-TK on ALFWorld and ToolBench using DeepSeek and Gemini Flash. Across 11,820 trajectories, rank watermarking preserves task utility relative to a clean AgentMark pipeline: average success is 78.6% versus 80.0% on ALFWorld and 77.1% versus 75.7% on ToolBench. Behavior-distribution divergence from clean remains comparable to symmetric AgentMark. Under exact probabilities, symmetric AgentMark has higher 8-bit payload recovery on ALFWorld, but under top-10 rank-only verification it collapses from 0.700 to 0.014 while AsymAgentMark-TK remains at 0.068. These results show that partial channel knowledge can be converted into practical behavioral provenance without requiring exact probability reconstruction.

I. INTRODUCTION

LLM systems are rapidly shifting from passive text generators to agents that perceive context, plan, call tools, and act across multiple steps [1], [2], [3]. This shift changes what provenance must mean. For a chatbot, attributing the final string may be enough. For an agent, the consequential object is often the trajectory: which tool it selected, which subgoal it pursued, which API it invoked, or which embodied action it chose. These planning behaviors can affect safety, accountability, intellectual-property protection, and post-hoc auditing even when the final natural-language output is short or absent.

Content watermarking for LLM outputs has made substantial progress [4], [5], but it does not directly solve behav-

ioral provenance. Planning behaviors are not token-native: an LLM’s reasoning text may be compiled into structured tool calls or environment actions, discarding the lexical signal used by token watermarks. In response, AgentMark [6] formulated behavioral watermarking as distribution-preserving sampling over an explicit per-step behavior distribution. Its concrete construction, which we call AgentMark-F, embeds multi-bit identifiers while preserving the marginal behavior distribution, and it tolerates partial logs through erasure coding.

However, AgentMark-F is symmetric: decoding requires the verifier to reconstruct the same probability distribution used by the embedder. This is a strong assumption in real deployments. The verifier may access a rounded top- k API, a model after provider-side updates, a different serving stack, or a proxy model used for audit. These changes often preserve the relative order of plausible behaviors while shifting the numeric probabilities. Symmetric distribution-preserving encoders are fragile under this mismatch because their decoding bins are probability-dependent. A small change in probabilities can alter bin boundaries and desynchronize decoding.

This paper asks whether behavioral provenance can survive with less verifier knowledge. We answer affirmatively by importing the principle of weakly asymmetric steganography [7] into LLM-agent behavior channels. In our setting, the embedder has the full behavior distribution P_t , but the verifier observes only partial channel knowledge: the top- k rank ordering σ_t . This setting is weaker than exact-probability verification but stronger than fully asymmetric verification with no channel knowledge. It matches common audit interfaces that expose candidate rankings without calibrated probabilities.

We present AsymAgentMark-TK, a top- k rank-based behavioral watermark. The key technical idea is to redesign the encoder and decoder together. The encoder sorts behaviors by probability and recursively partitions the top- k list into even and odd ranks. At each level, a binary distribution-preserving primitive uses probability masses to choose a group without changing the marginal policy. The decoder, by contrast, never uses probability values: the selected behavior’s rank determines the path through the same interleaved partition tree, and odd branches reveal embedded bits. A fixed-budget DRBG schedule ensures that encoder and decoder consume the same pseudorandomness even when no bit is embedded at a level.

Our contributions are:

- We formulate weakly asymmetric behavioral watermarking for LLM agents, separating the embedder’s full channel

access from the verifier’s top- k rank-only access.

- We design AsymAgentMark-TK, a distribution-preserving rank encoder/decoder that decodes from top- k ordering rather than exact probabilities.
- We integrate the construction with deterministic random linear network coding (RLNC), enabling payload recovery from variable-bit and partially observed trajectories.
- We evaluate on ALFWorld [8] and ToolBench [9] across DeepSeek and Gemini Flash models, measuring utility, stealth, rank-only decoding, top- k sensitivity, rank noise, and false-positive rates.

II. BACKGROUND AND MOTIVATION

A. Behavioral Watermarking for Agents

An LLM agent operates over a trajectory of observations, planning behaviors, and execution actions. At each step t , the agent chooses a behavior b_t from a finite admissible set $\mathcal{B}_t$. Examples include ALFWorld navigation actions, ToolBench API calls, or social-simulation behaviors in OASIS [10]. AgentMark models this choice as sampling from an elicited behavior distribution P_t and replaces ordinary sampling with a distribution-preserving encoder:

$$\hat{b}_t \leftarrow \text{Enc}(P_t, r_t), \quad \Pr[\hat{b}_t = b] = P_t(b). \quad (1)$$

The distribution-preservation requirement is central. Agent trajectories are long-horizon and stateful; even small per-step distribution shifts can compound into task drift, failures, or visibly abnormal behavior.

B. The Symmetry Bottleneck

The concrete AgentMark-F construction uses differential recombination over P_t followed by cyclic-shift encoding. This is efficient when both embedder and verifier share exact probabilities. The same property makes it brittle under probability perturbation: if the verifier’s reconstructed P'_t differs from P_t , recombined bins may differ, so the selected behavior is interpreted under the wrong bin structure.

The weakness is not merely numerical. Many verification interfaces intentionally withhold calibrated probabilities. Auditors may receive ranked candidates, top- k choices, or sampled logs. Even when probabilities are available, temperature scaling, quantization, floating-point differences, model upgrades, and fine-tuning can perturb values. These transformations frequently preserve a useful invariant: the order of high-probability behaviors.

This observation is especially natural for agents. Candidate behaviors are usually semantic alternatives produced by a planning prompt: a tool call, a navigation operation, a search/refine step, or a social action. A downstream auditor may be able to ask a model to rank plausible next behaviors from the same logged context even if the exact sampling distribution is hidden. In this sense, behavior-level verification has a useful middle ground that token-level watermarking often lacks: ranks are easier to reproduce than calibrated probabilities, and more informative than the selected action alone.

C. Weak Asymmetry

Provably secure steganography traditionally separates symmetric settings, where sender and receiver share channel distributions, from asymmetric settings, where the receiver has no channel oracle [11], [12]. Weak asymmetry occupies the middle ground: the sender has the full channel, while the receiver has partial but nontrivial channel knowledge [7]. We instantiate this idea for behavioral watermarking. The embedder uses P_t to preserve the agent policy; the verifier uses only $\sigma_t = \text{argsort}_k(P_t)$ to decode.

III. PROBLEM FORMULATION

A. Agent Behavior Channel

For a trajectory of length T , each step t has a behavior set $\mathcal{B}_t = \{b_{t,1}, \dots, b_{t,n_t}\}$ and a behavior distribution P_t induced by the agent and environment context. A watermarking key K_{sh} and step context h_t derive per-step pseudorandomness. The embedder encodes a payload $m \in \{0,1\}^L$ into selected behaviors $\hat{b}_1, \dots, \hat{b}_T$ while preserving P_t at each step.

The verifier observes a possibly incomplete set of indices $I \subseteq \{1, \dots, T\}$ and, for each observed step, the selected behavior $\hat{b}_t$ plus channel side information. In the symmetric setting, the side information is P_t or an exact reconstruction. In our setting, it is the top- k ordering σ_t .

B. Adversary and Requirements

We consider three perturbations. First, *step erasure*: some logged steps are missing due to execution failures, platform deletion, or incomplete telemetry. Second, *distribution perturbation*: a verifier reconstructs P'_t whose numeric values differ from P_t . Third, *rank noise*: some adjacent ranks in σ_t are swapped, modeling top- k instability or proxy-model disagreement.

A scheme should satisfy:

- **Utility preservation:** $\Pr[\hat{b}_t = b] = P_t(b)$ per step.
- **Weak-asymmetric correctness:** if $\text{argsort}_k(P'_t) = \text{argsort}_k(P_t)$, decoding from P'_t ’s order returns the same bits as decoding from P_t ’s order.
- **Low false positives:** a clean trajectory should match a claimed L -bit payload with probability approximately 2^{-L} under independent payload trials.
- **Erasure tolerance:** payload recovery should degrade gracefully when only a subset of steps is observed.

C. Non-Goals

AsymAgentMark-TK is not a general-purpose watermark removal defense. If an adversary controls the agent and can arbitrarily rewrite the trajectory, no behavioral watermark can force attribution without an external logging root of trust. Our target is post-hoc verification for trajectories that are generated by an honest embedder but later audited through an imperfect channel interface. We also do not assume the verifier can re-run the exact original model. The verifier may use a same-family model, a refreshed endpoint, or a restricted API as long as the top- k order remains sufficiently stable.

TABLE I
DECODER INFORMATION REQUIREMENTS.

Scheme	Encoder input	Decoder input
AgentMark-F	exact P_t	exact P_t
AsymAgentMark-TK	exact P_t	top- k rank σ_t

IV. DESIGN

A. Overview

AsymAgentMark-TK has four components. First, the agent elicits or estimates P_t over candidate behaviors, following AgentMark’s black-box-compatible behavior-distribution interface [6]. Second, the encoder sorts behaviors by descending probability and restricts embedding to the top- k region. Third, recursive interleaved binary splitting embeds bits while preserving the behavior distribution. Fourth, deterministic RLNC maps the payload into a stream of coded bits so that the verifier can solve for the payload after observing enough independent coded bits.

B. Binary Distribution-Preserving Primitive

At a tree node, the encoder must choose between two groups with masses S_0 and S_1 while optionally embedding one bit. Assume $S_0 \geq S_1$ and $S = S_0 + S_1$. Given message bit $m \in \{0, 1\}$ and $r \leftarrow U[0, 1)$ from the DRBG, define

$$r_m = (rS + \frac{1}{2}Sm) \bmod S. \quad (2)$$

BinEnc selects group 0 if $r_m < S_0$ and group 1 otherwise. If group 1 is selected, one bit is embedded; if group 0 is selected, no bit is consumed and the same bit is retried at the next level. Because r_m is uniform over $[0, S)$ for uniform r , the selected group has exactly the desired marginal probabilities S_0/S and S_1/S .

C. Rank Encoder

Let σ_t be the descending rank ordering of P_t . The encoder first samples whether to enter the top- k region with probability $S_{\text{top}} = \sum_{i=1}^k P_t(\sigma_t(i))$. If not, it samples from the tail proportionally and embeds no bits. Inside top- k , it normalizes probabilities and applies recursive splitting:

- 1) Split the current sorted list into even ranks $(0, 2, 4, \dots)$ and odd ranks $(1, 3, 5, \dots)$.
- 2) Use BinEnc on the two group masses.
- 3) Recurse into the selected group until a single behavior remains.

Interleaving is important. A top-half/bottom-half split would place most mass in the first group, producing low embedding opportunity. Even/odd splitting balances mass while preserving a rank path that the verifier can recover.

D. Rank Decoder

Given selected behavior $\hat{b}_t$ and top- k order σ_t , the decoder finds the zero-indexed rank q of $\hat{b}_t$. If $\hat{b}_t$ is outside top- k , the

Algorithm 1: One-step rank encoding.

Input: distribution P_t , context h_t , key K_{sh} , payload stream M , pointer ℓ , top- k .
1. Derive DRBG $\mathcal{G}_t$ from (K_{sh}, h_t) .
2. Sort behaviors: $\sigma_t \leftarrow \text{argsort}(P_t)$.
3. With probability $1 - \sum_{i \leq k} P_t(\sigma_t(i))$, sample from the tail and return no bits.
4. Normalize the top- k distribution D .
5. While $|D| > 1$: split D into even/odd ranks; run BinEnc on the group masses; consume a message bit only when BinEnc enters the odd group; recurse into the chosen group.
6. Advance unused DRBG calls until exactly $\lceil \log_2 k \rceil$ calls are consumed.
Output: selected behavior and consumed coded bits.

Fig. 1. Encoder sketch. The encoder uses probability values only to preserve the marginal behavior distribution.

Algorithm 2: One-step rank decoding.

Input: selected behavior $\hat{b}_t$, top- k order σ_t , context h_t , key K_{sh} .
1. Derive the same DRBG sequence $r_0, \dots, r_{\lceil \log_2 k \rceil - 1}$.
2. If $\hat{b}_t \notin \sigma_t[1 : k]$, return the empty string.
3. Let q be the zero-indexed rank of $\hat{b}_t$.
4. While $q \neq 0$: if q is odd, append 1 when the corresponding $r_i < 0.5$ and 0 otherwise; set $q \leftarrow \lfloor q/2 \rfloor$.
Output: decoded coded bits for step t .

Fig. 2. Decoder sketch. No numeric probability appears in the decoder.

step yields no bits. Otherwise, the decoder reconstructs the path by repeatedly reading the parity of q :

$$q_0 = q, \quad q_{\ell+1} = \lfloor q_\ell/2 \rfloor. \quad (3)$$

Whenever q_ℓ is odd, the selected path entered the odd group and one bit was embedded. The value of that bit is determined by the same DRBG value used at level ℓ : the implementation decodes it as 1 if $r_\ell < 0.5$ and 0 otherwise. The decoder consumes exactly $\lceil \log_2 k \rceil$ DRBG values per step, matching the encoder’s fixed synchronization budget.

E. Coding Layer

Per-step embedding capacity varies with P_t , k , and trajectory length. To avoid requiring a contiguous bitstream, AsymAgentMark-TK uses deterministic RLNC over GF(2). For payload $m \in \{0, 1\}^L$, the i -th coded bit is

$$c_i = \langle a_i, m \rangle \bmod 2, \quad (4)$$

where coefficient vector a_i is generated deterministically from a stream key and index i . The verifier collects decoded coded bits and solves the resulting linear system by Gaussian elimination over GF(2). This layer is inherited from AgentMark’s erasure-resilience design but is independent of whether the step decoder is symmetric or rank-only.

F. Implementation Notes

The implementation follows the repository’s production path. The public SDK exposes an AgentWatermarker wrapper with `algorithm="rank"` as the default. Internally, `Dist` stores probability tensors and stable indices, `RankEncStep` applies the interleaved split, and `RankDecStep` reconstructs bits from sorted indices. The

DRBG is an HMAC-SHA512 reseed construction inspired by Meteor [12]. A synchronization counter, `rt_sync`, forces both sides to advance exactly $\lceil \log_2 n \rceil$ random draws per step. This detail is small but critical: without it, a zero-bit level would shift all later decoding.

Tie handling uses stable sorting after probability normalization. Ties are rare in model-scored behavior sets but can occur after rounding or prompt templates that assign equal fallback scores. Stable sorting makes the order deterministic when both sides see the same candidate list. If the verifier obtains a tied but differently ordered top- k list, this is treated as rank noise and measured through the adjacent-swap experiment.

V. SECURITY ANALYSIS

Proposition 1 (distribution preservation). For each step t and behavior $b \in \mathcal{B}_t$, AsymAgentMark-TK outputs b with probability $P_t(b)$.

Proof sketch. The top/tail gate selects the top- k region with probability equal to its total mass and otherwise samples the tail proportionally. Conditioned on entering top- k , each binary split uses BinEnc, whose selected group probability equals its normalized mass because r_m is uniform. Induction over the partition tree gives each leaf its normalized probability within top- k . Multiplying by the top- k mass recovers $P_t(b)$. $\square$

Theorem 1 (weak asymmetry). Let P_t and P'_t satisfy $\text{argsort}_k(P_t) = \text{argsort}_k(P'_t)$. For any selected behavior $\hat{b}_t$ and shared context-derived DRBG stream,

$$\text{RankDec}(\hat{b}_t, \text{argsort}_k(P_t)) = \text{RankDec}(\hat{b}_t, \text{argsort}_k(P'_t)). \quad (5)$$

Proof. RankDec accesses channel information only through the rank position of $\hat{b}_t$ in the top- k order. If the two orders are equal, this rank is identical. The DRBG stream depends on the secret key and context, not probability values. Therefore the decoded path and bit values are identical. $\square$

False positives. For an unwatermarked trajectory, a claimed L -bit payload matches by chance with probability 2^{-L} under the standard assumption that decoded clean bits are independent of the claimed payload. In practice the verifier can tune L to the desired audit threshold; $L = 32$ yields 2.33×10^{-10} and $L = 64$ yields 5.42×10^{-20} .

Limits. AsymAgentMark-TK is robust to value perturbations that preserve top- k order, not to arbitrary rank manipulation. If an adversary can force targeted rank swaps in the top- k list, it can corrupt decoded paths. This is the expected boundary of any rank-only weakly asymmetric scheme. The evaluation therefore includes adjacent rank-noise injection.

VI. EVALUATION

A. Setup

We evaluate on ALFWorld [8] and ToolBench [9]. ALFWorld tests embodied planning over ID and OOD splits; ToolBench tests tool-use behavior across six split categories. We use DeepSeek [13] and Gemini Flash [3]. The postprocessed corpus contains 11,820 trajectories and 4,728 watermark decode-capable trajectories. Methods are: vanilla LLM

TABLE II
TASK UTILITY AVERAGED OVER AVAILABLE MODEL/SPLIT GROUPS.

Dataset	Method	Success	Steps	Groups
ALFWorld	Vanilla	69.1	16.54	4
	Clean	80.0	16.23	4
	RG	74.9	17.58	4
	AgentMark-F	79.4	16.58	4
	AsymAgentMark-TK	78.6	16.40	4
ToolBench	Vanilla	84.1	4.06	2
	Clean	75.7	5.32	2
	RG	75.1	4.78	2
	AgentMark-F	74.7	4.54	2
	AsymAgentMark-TK	77.1	5.07	2

agent, clean AgentMark pipeline without embedding, red-green (RG) behavior bias, symmetric AgentMark-F, and rank-based AsymAgentMark-TK.

Unless otherwise stated, recovery rates in this draft are computed from the available offline decoder artifacts. Capacity tables are bit-level channel proxies over logged trajectories and should not be conflated with a separate end-to-end RLNC recovery run.

B. Research Questions

The evaluation answers five questions. **RQ1:** Does rank-based embedding preserve task utility compared with clean and symmetric AgentMark pipelines? **RQ2:** Does it leave behavior-frequency distributions close to clean trajectories? **RQ3:** What is the tradeoff between exact-probability capacity and rank-only robustness? **RQ4:** How sensitive is the method to the top- k parameter? **RQ5:** How quickly does decoding degrade when the verifier’s rank order is noisy?

C. Utility Preservation

Table II reports average task success and steps. On ALFWorld, AsymAgentMark-TK reaches 78.6% success versus 80.0% for clean and 79.4% for AgentMark-F. On ToolBench, AsymAgentMark-TK reaches 77.1%, slightly above the clean pipeline’s 75.7%. The result supports the distribution-preservation goal: rank encoding does not introduce the large utility tax associated with naive behavior bias.

D. Behavioral Stealth

We compare empirical action distributions against clean trajectories using Jensen-Shannon divergence (JSD). On ALFWorld, mean JSD to clean is 0.121 for AsymAgentMark-TK and 0.122 for AgentMark-F; on ToolBench it is 0.140 for AsymAgentMark-TK and 0.174 for AgentMark-F. RG is comparable on ALFWorld but worse on ToolBench. These measurements are coarse because behavior distributions are task- and split-dependent, but they show no obvious distributional anomaly introduced by rank splitting.

E. Rank-Only Verification

Table IV compares payload recovery and bit match for 8-bit payloads under different verifier knowledge. Under exact probabilities on ALFWorld, AgentMark-F has higher payload

TABLE III
MEAN JSD TO CLEAN BEHAVIOR FREQUENCY. LOWER IS CLOSER TO CLEAN.

Dataset	AgentMark-F	AsymAgentMark-TK	RG	Vanilla
ALFWorld	0.122	0.121	0.134	0.564
ToolBench	0.174	0.140	0.181	0.162

TABLE IV
8-BIT RECOVERY UNDER VERIFIER CHANNEL KNOWLEDGE. CELLS SHOW PAYLOAD RECOVERY / BIT-MATCH RATE.

Dataset	Method	Exact	Top-10	Top-6	Top-2
ALFWorld	AgentMark-F	.700/.985	.014/.594	.004/.498	.004/.504
	AsymAgentMark-TK	.532/.942	.068/.718	.028/.641	.006/.566
ToolBench	AgentMark-F	.085/.582	.003/.299	.000/.292	.000/.309
	AsymAgentMark-TK	.033/.502	.032/.501	.007/.470	.000/.357

recovery (0.700) than AsymAgentMark-TK (0.532), reflecting the efficiency cost of rank-only structure. Under top-10 rank-only verification, AgentMark-F collapses to 0.014 payload recovery, while AsymAgentMark-TK remains at 0.068. This is the central tradeoff: AgentMark-F is better when exact probabilities are available; AsymAgentMark-TK is more reliable when the verifier only has rank knowledge.

ToolBench is lower-capacity for both schemes because trajectories are short and many tool choices have low recoverable entropy. Nevertheless, AsymAgentMark-TK remains stable from exact probabilities to top-10 rank-only decoding (0.033 to 0.032), whereas AgentMark-F drops from 0.085 to 0.003.

F. Top- k Ablation

Increasing k raises both the amount of available rank information and the risk of rank instability. In ALFWorld, AsymAgentMark-TK’s mean bit-match rate rises from 0.558 at $k = 2$ to 0.863 at $k = 20$, with payload recovery increasing from 0.006 to 0.313. ToolBench improves more modestly, from 0.351 to 0.505 bit-match, reflecting lower behavior entropy and shorter horizons. These results suggest that k should be selected per environment: larger k helps high-entropy embodied planning, while low-entropy tool tasks require either longer traces, stronger coding, or adaptive gating.

G. Rank Noise

We inject adjacent rank swaps with noise rate ϵ . On ALFWorld, AsymAgentMark-TK maintains a bit-match rate of 0.900 at $\epsilon = 0.05$ and 0.858 at $\epsilon = 0.10$, degrading to 0.688 at $\epsilon = 0.30$. ToolBench starts near chance because of lower decoded length and capacity, but its degradation is smooth. The experiment confirms the expected boundary: rank-only decoding is insensitive to probability values but sensitive to rank swaps.

H. False Positives

Analytic clean-trajectory trials match the expected false-positive probability 2^{-L} . For payload lengths $L = 8, 16, 32, 64$, the expected rates are 3.91×10^{-3} , 1.53×10^{-5} , 2.33×10^{-10} , and 5.42×10^{-20} . Operational deployments

TABLE V
TOP- k ABLATION FOR ASYAGENTMARK-TK. CELLS SHOW BIT-MATCH / PAYLOAD RECOVERY.

Dataset	$k = 2$	$k = 4$	$k = 6$	$k = 8$	$k = 10$	$k = 20$
ALFWorld	.558/.006	.601/.013	.642/.028	.670/.048	.707/.064	.863/.313
ToolBench	.351/.000	.426/.006	.470/.010	.488/.021	.495/.032	.505/.035

TABLE VI
RANK-NOISE SENSITIVITY FOR ASYAGENTMARK-TK. CELLS SHOW BIT-MATCH / PAYLOAD RECOVERY.

Dataset	0.00	0.05	0.10	0.20	0.30
ALFWorld	.942/.532	.900/.368	.858/.239	.769/.108	.688/.042
ToolBench	.502/.033	.489/.025	.454/.017	.413/.003	.375/.003

should use at least 32-bit claims for audit-grade attribution, with longer payloads when enough trajectory length is available.

VII. DISCUSSION

Exact probability is a luxury. The evaluation supports a pragmatic conclusion: if exact P_t is available, symmetric AgentMark-F can be more capacity-efficient. If the verifier has only top- k ranks, symmetric decoding becomes unreliable and weak asymmetry becomes necessary.

ToolBench exposes a low-entropy regime. Tool-use traces are shorter and often contain fewer semantically viable actions than embodied planning. AsymAgentMark-TK preserves utility in this setting, but the current offline payload recovery is low. This suggests combining rank encoding with adaptive step selection, longer audit windows, or higher-level tool-plan aggregation.

Rank consistency is observable. Unlike exact probability equality, rank consistency can be measured directly during verification. A verifier can report top- k agreement, Kendall tau distance, or adjacent-swap rates to calibrate confidence.

Adaptive adversaries. An adversary aware of the watermark could try to perturb ranks while preserving task success. This attack is outside pure distribution perturbation and should be treated as an active removal attack. Defenses include randomized k , context-dependent rank partitions, and cross-step consistency checks.

VIII. RELATED WORK

LLM and agent watermarking. Token-level LLM watermarking modifies or tests generated text distributions [4], [5]. These methods are complementary to behavioral watermarking because agents often compile natural-language plans into structured actions. AgentMark [6] is the closest prior work and establishes distribution-preserving behavioral watermarking. AsymAgentMark-TK keeps its utility-preservation objective but weakens the verifier’s channel requirement.

Steganography and distribution-preserving sampling. Provably secure steganography formalizes indistinguishability between cover and stego content [11]. Meteor provides a practical cryptographic construction for realistic distributions [12].

Weakly asymmetric steganography studies receivers with partial channel knowledge [7]. AsymAgentMark-TK adapts this idea from token/text channels to planning-behavior channels.

Agent benchmarks. ALFWorld aligns text-based embodied environments with interactive learning [8]. ToolBench evaluates LLM tool-use over real-world APIs [9]. OASIS demonstrates large-scale social-agent simulation [10]. These benchmarks highlight why behavior-level provenance matters: the action trajectory is often more important than the final message.

IX. LIMITATIONS AND ETHICS

This work is intended for provenance, audit, and authorized ownership verification. It should not be used to conceal malicious coordination or bypass platform policy. The construction assumes access to a behavior distribution at embedding time and top- k rank information at verification time. It does not protect against arbitrary rank manipulation, model-level watermark removal, or compromised keys. The current evaluation uses offline decoder artifacts; a final deployment study should include end-to-end RLNC recovery under live logging and cross-model rank reconstruction.

X. CONCLUSION

Behavioral provenance for LLM agents should not depend on a verifier reproducing exact per-step probabilities. AsymAgentMark-TK shows that top- k rank order is enough to recover useful watermark signal while preserving the agent’s marginal behavior policy. By replacing probability-dependent bins with a rank-decodable binary partition tree, the scheme trades some exact-channel capacity for robustness under realistic partial-channel verification. This weakly asymmetric view makes behavioral watermarking better aligned with how agents are actually audited.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [2] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, “GPT-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [3] Gemini Team, “Gemini: A family of highly capable multimodal models,” *arXiv preprint arXiv:2312.11805*, 2023.
- [4] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, “A watermark for large language models,” in *Proceedings of the International Conference on Machine Learning*, 2023.
- [5] S. Dathathri, A. See, S. Ghaisas, P.-S. Huang, R. McAdam, J. Welbl, V. Bachani, A. Kaskasoli, R. Stanforth, T. Matejovicova, J. Hayes, N. Vyas, T. Zaccary, M. Zilka, I. Gabriel, W. Isaac, A. Guez, L. Weidinger, J. Mellor, D. Hassabis, K. Kavukcuoglu, G. Irving, and P. Kohli, “Scalable watermarking for identifying large language model outputs,” *Nature*, 2024.
- [6] K. Huang, J. Tan, Y. Wei, W. Li, Z. Zhang, H. Tian, Z. Yang, and L. Zhou, “AgentMark: Utility-preserving behavioral watermarking for agents,” in *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2026.
- [7] Anonymous, “From symmetry toward weak asymmetry: Secure steganography under the receiver’s partial channel knowledge,” *Manuscript*, 2026.
- [8] M. Shridhar, X. Yuan, M.-A. Côté, Y. Bisk, A. Trischler, and M. Hausknecht, “ALFWorld: Aligning text and embodied environments for interactive learning,” in *Proceedings of the International Conference on Learning Representations*, 2021.
- [9] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, “ToolLLM: Facilitating large language models to master 16000+ real-world apis,” *arXiv preprint arXiv:2307.16789*, 2023.
- [10] C. Gao, X. Lan, Z. Lu, J. Mao, J. Piao, H. Wang, D. Jin, and Y. Li, “OASIS: Open agents social interaction simulations with one million agents,” *arXiv preprint arXiv:2411.11581*, 2024.
- [11] N. J. Hopper, J. Langford, and L. von Ahn, “Provably secure steganography,” in *Proceedings of the Annual International Cryptology Conference*, 2002.
- [12] G. Kaptchuk, T. M. Jois, M. Green, and A. D. Rubin, “Meteor: Cryptographically secure steganography for realistic distributions,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2021.
- [13] DeepSeek-AI, “DeepSeek llm: Scaling open-source language models with longtermism,” *arXiv preprint arXiv:2401.02954*, 2024.